One of the taxi suppliers has agreed to participate in this program. Others are likely to follow. Still others may decline.

Of course, some drivers may be allergic to cats, so those drivers should never be selected for this service. Also, some customers will undoubtedly have similar allergies, so a vehicle that has been used to deliver kittens within the last 3 days should not be selected for customers who declare such allergies.

Look at that diagram of services. How many of those services will have to change to implement this feature? *All of them*. Clearly, the development and deployment of the kitty feature will have to be very carefully coordinated.

In other words, the services are all coupled, and cannot be independently developed, deployed, and maintained.

This is the problem with cross-cutting concerns. Every software system must face this problem, whether service oriented or not. Functional decompositions, of the kind depicted in the service diagram in Figure 27.1, are very vulnerable to new features that cut across all those functional behaviors.

OBJECTS TO THE RESCUE

How would we have solved this problem in a component-based architecture? Careful consideration of the SOLID design principles would have prompted us to create a set of classes that could be polymorphically extended to handle new features.

The diagram in Figure 27.2 shows the strategy. The classes in this diagram roughly correspond to the services shown in Figure 27.1. However, note the boundaries. Note also that the dependencies follow the Dependency Rule.

Much of the logic of the original services is preserved within the base classes of the object model. However, that portion of the logic that was specific to *rides* has been extracted into a `Rides` component. The new feature for kittens has been placed into a `Kittens` component. These two components override the abstract base classes in the original components using a pattern such as *Template Method or Strategy*.

Note again that the two new components, `Rides` and `Kittens`, follow the Dependency Rule. Note also that the classes that implement those features are created by factories under the control of the UI.

Clearly, in this scheme, when the Kitty feature is implemented, the `TaxiUI` must change. But nothing else needs to be changed. Rather, a new jar file, or Gem, or DLL is added to the system and dynamically loaded at runtime.

Thus the Kitty feature is decoupled, and independently developable and deployable.

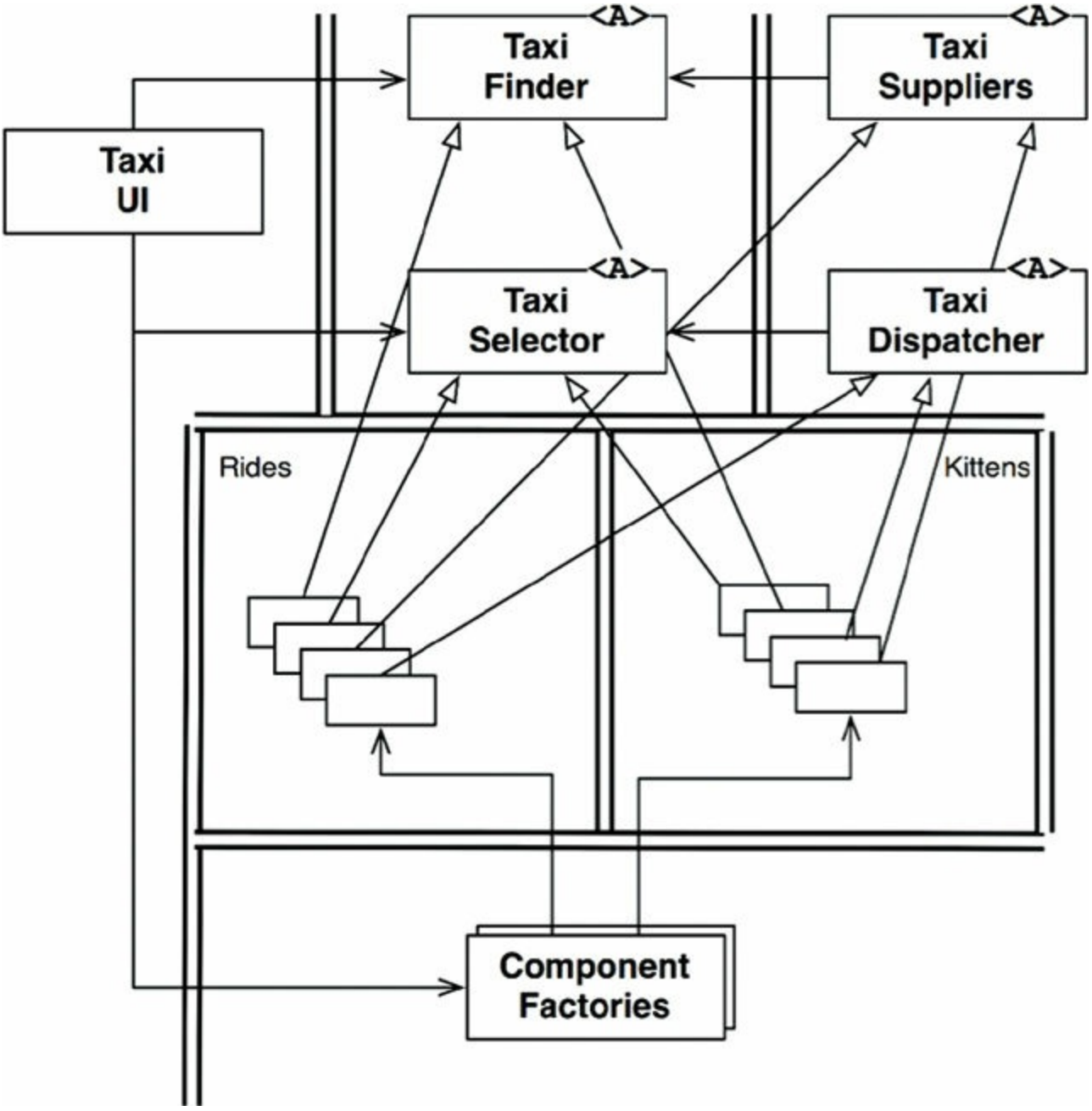

Figure 27.2 Using an object-oriented approach to deal with cross-cutting concerns

COMPONENT-BASED SERVICES

The obvious question is: Can we do that for services? And the answer is, of course: Yes! Services do not need to be little monoliths. Services can, instead, be designed using the SOLID principles, and given a component structure so that new components can be added to them without changing the existing components within the service.

Think of a service in Java as a set of abstract classes in one or more jar files. hink of each new feature or feature extension as another jar file that contains classes that extend the abstract classes in the first jar files. Deploying a new feature then becomes not a matter of redeploying the services, but rather a matter of simply *adding* the new jar files to the load paths of those services. In other words, adding new features conforms to the Open-Closed Principle.

The service diagram in Figure 27.3 shows the structure. The services still exist as before, but each has its own internal component design, allowing new features to be added as new derivative classes. Those derivative classes live within their own components.

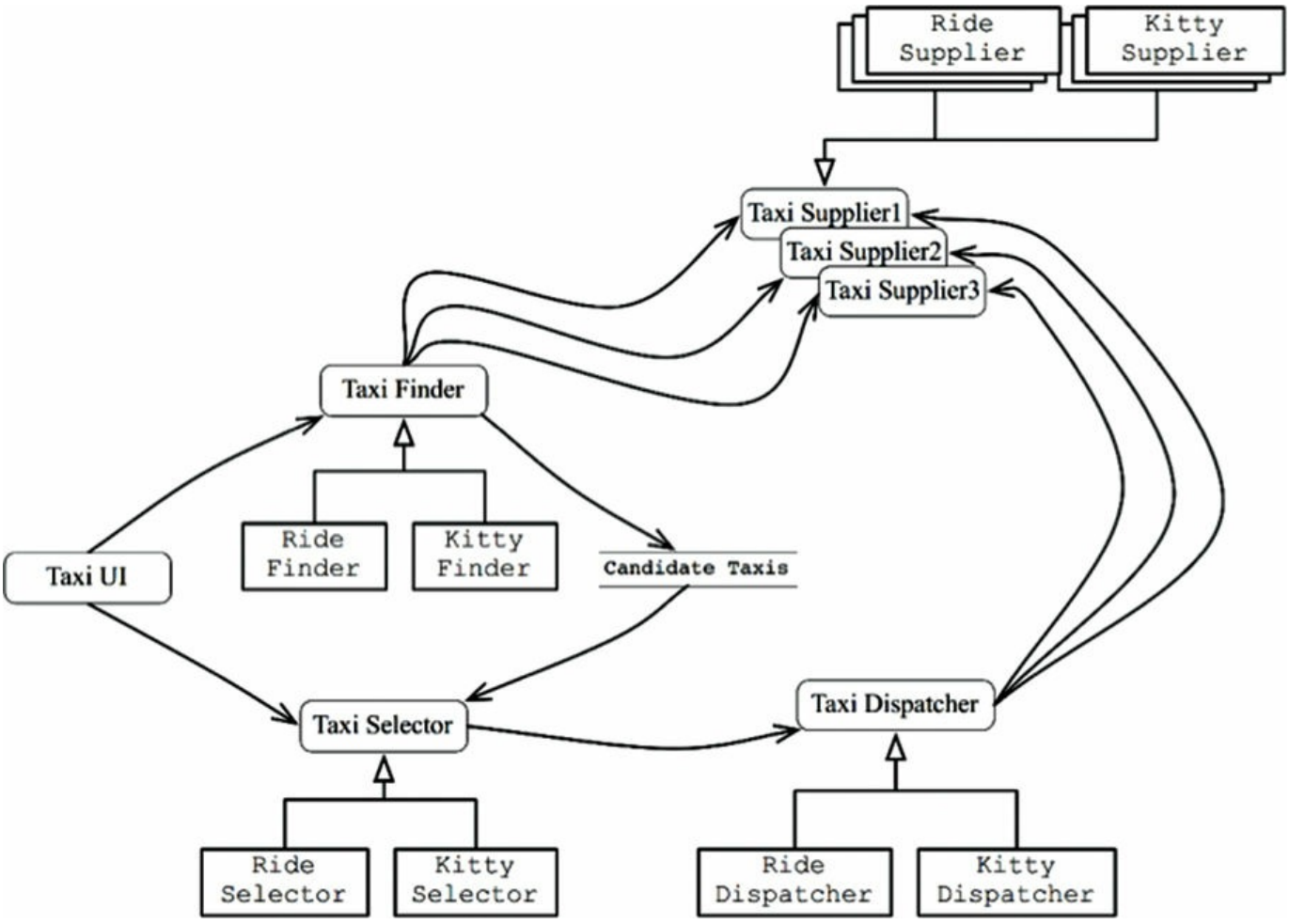

Figure 27.3 Each service has its own internal component design, enabling new features to be added as new derivative classes

CROSS-CUTTING CONCERNS

What we have learned is that architectural boundaries do not fall *between* services. Rather, those boundaries run *through* the services, dividing them into components.

To deal with the cross-cutting concerns that all significant systems face, services must be designed with internal component architectures that follow the Dependency Rule, as shown in the diagram in Figure 27.4. Those services do not define the architectural boundaries of the system; instead, the components within the services do.

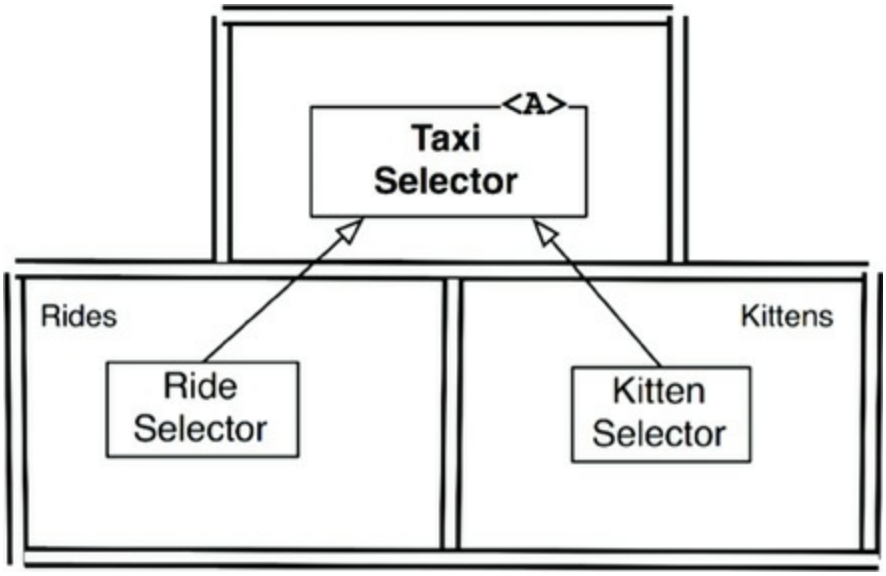

Figure 27.4 Services must be designed with internal component architectures that follow the Dependency Rule

CONCLUSION

As useful as services are to the scalability and develop-ability of a system, they are not, in and of themselves, architecturally significant elements. The architecture of a system is defined by the boundaries drawn within that system, and by the dependencies that cross those boundaries. That architecture is not defined by the physical mechanisms by which elements communicate and execute.

A service might be a single component, completely surrounded by an architectural boundary. Alternatively, a service might be composed of several components separated by architectural boundaries. In rare[2] cases, clients and services may be so coupled as to have no architectural significance whatever.

1. Therefore the number of micro-services will be roughly equal to the number of programmers.
2. We hope they are rare. Unfortunately, experience suggests otherwise.

28
THE TEST BOUNDARY

Yes, that's right: *The tests are part of the system*, and they participate in the architecture just like every other part of the system does. In some ways, that participation is pretty normal. In other ways, it can be pretty unique.

TESTS AS SYSTEM COMPONENTS

There is a great deal of confusion about tests. Are they part of the system? Are they separate from the system? Which kinds of tests are there? Are unit tests and integration tests different things? What about acceptance tests, functional tests, Cucumber tests, TDD tests, BDD tests, component tests, and so on?

It is not the role of this book to get embroiled in that particular debate, and

fortunately it isn't necessary. From an architectural point of view, all tests are the same. Whether they are the tiny little tests created by TDD, or large FitNesse, Cucumber, SpecFlow, or JBehave tests, they are architecturally equivalent.

Tests, by their very nature, follow the Dependency Rule; they are very detailed and concrete; and they always depend inward toward the code being tested. In fact, you can think of the tests as the outermost circle in the architecture. Nothing within the system depends on the tests, and the tests always depend inward on the components of the system.

Tests are also independently deployable. In fact, most of the time they are deployed in test systems, rather than in production systems. So, even in systems where independent deployment is not otherwise necessary, the tests will still be independently deployed.

Tests are the most isolated system component. They are not necessary for system operation. No user depends on them. Their role is to support development, not operation. And yet, they are no less a system component than any other. In fact, in many ways they represent the model that all other system components should follow.

DESIGN FOR TESTABILITY

The extreme isolation of the tests, combined with the fact that they are not usually deployed, often causes developers to think that tests fall outside of the design of the system. This is a catastrophic point of view. Tests that are not well integrated into the design of the system tend to be fragile, and they make the system rigid and difficult to change.

The issue, of course, is coupling. Tests that are strongly coupled to the system must change along with the system. Even the most trivial change to a system component can cause many coupled tests to break or require changes.

This situation can become acute. Changes to common system components can cause hundreds, or even thousands, of tests to break. This is known as the *Fragile Tests Problem*.

It is not hard to see how this can happen. Imagine, for example, a suite of tests that use the GUI to verify business rules. Such tests may start on the login screen and then navigate through the page structure until they can check particular business

rules. Any change to the login page, or the navigation structure, can cause an enormous number of tests to break.

Fragile tests often have the perverse effect of making the system rigid. When developers realize that simple changes to the system can cause massive test failures, they may resist making those changes. For example, imagine the conversation between the development team and a marketing team that requests a simple change to the page navigation structure that will cause 1000 tests to break.

The solution is to design for testability. The first rule of software design—whether for testability or for any other reason—is always the same: *Don't depend on volatile things*. GUIs are volatile. Test suites that operate the system through the GUI *must be fragile*. Therefore design the system, and the tests, so that business rules can be tested without using the GUI.

THE TESTING API

The way to accomplish this goal is to create a specific API that the tests can use to verify all the business rules. This API should have superpowers that allow the tests to avoid security constraints, bypass expensive resources (such as databases), and force the system into particular testable states. This API will be a superset of the suite of *interactors* and *interface adapters* that are used by the user interface.

The purpose of the testing API is to decouple the tests from the application. This decoupling encompasses more than just detaching the tests from the UI: The goal is to decouple the *structure* of the tests from the *structure* of the application.

STRUCTURAL COUPLING

Structural coupling is one of the strongest, and most insidious, forms of test coupling. Imagine a test suite that has a test class for every production class, and a set of test methods for every production method. Such a test suite is deeply coupled to the structure of the application.

When one of those production methods or classes changes, a large number of tests must change as well. Consequently, the tests are fragile, and they make the production code rigid.

The role of the testing API is to hide the structure of the application from the tests.